



TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico 3

July 21, 2025

- Juan Claros - 111254
- Yoel Gaston Arlia - 111520
- Guido Cuppari - 111172
- Josue Daniel Mamani - 111514

Índice

1	<i>Problema 2</i>	3
1.1	Supuestos	3
1.2	Diseño	3
1.3	Pseudocódigo	3
1.4	Análisis de Complejidad	4
1.5	Seguimiento	4
1.6	Sets de Datos	4
1.7	Tiempos de Ejecución	5
1.8	Resultados	5
2	<i>Problema 3</i>	6
2.1	Enunciado	6
2.2	Descripción de un autómata finito determinístico para cada uno de los lenguajes simples	6
2.3	Descripción formal de los autómatas finitos determinísticos de los lenguajes A y B	7
2.4	Representación gráfica de los seis autómatas	8
2.5	Ejemplos de cadenas que son aceptadas y otras rechazadas para cada uno de los autómatas	11

1 Problema 2

1.1 Supuestos

- El numero total de ofertas n es conocido de antemano por el vendedor.
- Las ofertas se presentan en un orden aleatorio y no pueden ser reordenadas.
- Cada oferta es distinta, por lo tanto, existe una unica oferta máxima.
- Una vez que se rechaza una oferta, no puede recuperarse ni reconsiderarse.
- El vendedor puede comparar las ofertas vistas hasta el momento, pero no tiene información sobre las que vendran.
- Se asume que n es impar, lo cual permite simplificar el análisis probabilístico.

1.2 Diseño

El algoritmo implementa una estrategia inspirada en el clásico problema del mejor candidato. Se divide en dos fases:

1. **Fase de observación:** se descartan las primeras $r = \lfloor n/2 \rfloor$ ofertas, observando cuál es la mejor entre ellas.
2. **Fase de selección:** se acepta la primera oferta que supere a la mejor observada. Si ninguna lo hace, se acepta la última.

Esta estrategia asegura una probabilidad de al menos $\frac{1}{4}$ de seleccionar la mejor oferta del conjunto, sin importar el valor de n (siempre que sea impar y razonablemente grande).

Estructuras de datos utilizadas

- **Lista de enteros:** contiene las n ofertas en orden aleatorio.
- **Variable entera mejorObservada:** guarda la mayor oferta observada durante la primera mitad del proceso.

1.3 Pseudocodigo

```

1 Entrada: lista de ofertas aleatoriamente ordenadas de tamaño n
2 Salida: oferta seleccionada por el vendedor
3
4 1.  $r \leftarrow \text{floor}(n / 2)$ 
5 2.  $\text{mejorObservada} \leftarrow \max(\text{ofertas}[0 \dots r-1])$ 
6 3. Para  $i$  desde  $r$  hasta  $n - 1$  hacer:
7     Si  $\text{ofertas}[i] > \text{mejorObservada}$  entonces
8         retornar  $\text{ofertas}[i]$ 
9 4. Retornar  $\text{ofertas}[n - 1]$  // si ninguna supera a la mejor
   observada

```

1.4 Analisis de Complejidad

- La operación de calcular el máximo entre las primeras r ofertas tiene complejidad $O(n)$.
- La segunda fase también puede recorrer hasta n elementos, en el peor caso.
- Por lo tanto, la complejidad total del algoritmo es $O(n)$.

1.5 Seguimiento

- Supongamos que $n = 7$ y que las ofertas llegan en el siguiente orden aleatorio:

Ofertas = [30, 60, 25, 40, 85, 70, 50]

- Fase de observación: $r = \lfloor 7/2 \rfloor = 3$

Ofertas observadas: [30, 60, 25] $\Rightarrow$ mejorObservada = 60

- Fase de selección:
 - $40 < 60 \rightarrow$ rechazada
 - $85 > 60 \rightarrow$ **aceptada**
- Resultado: el vendedor acepta la oferta de **85**, que es la máxima del conjunto.

1.6 Sets de Datos

Para evaluar experimentalmente el comportamiento del algoritmo, se generaron conjuntos de datos aleatorios utilizando una función `random.shuffle()` con semilla fija, lo cual garantiza la **reproducibilidad** de los resultados.

- Para cada tamaño $n \in \{100, 500, 1000, 5000, 10000, 20000, 30000, 40000, 50000\}$ se generaron 100 sets distintos de ofertas.
- Cada set es una permutación aleatoria de los enteros del 1 al n .
- Se guardó uno de los sets de cada tamaño en el archivo `setdedatos.txt`.
- El archivo `resultados.txt` contiene los resultados de todas las ejecuciones:
 - Tamaño del set n
 - Semilla usada
 - Oferta seleccionada por el algoritmo
 - Máxima oferta del set
 - Si se seleccionó o no la mejor oferta

1.7 Tiempos de Ejecución

Se midió el **tiempo de ejecución promedio** del algoritmo para cada valor de n mencionado anteriormente, repitiendo la prueba 100 veces por tamaño y dividiendo el tiempo total por la cantidad de ejecuciones.

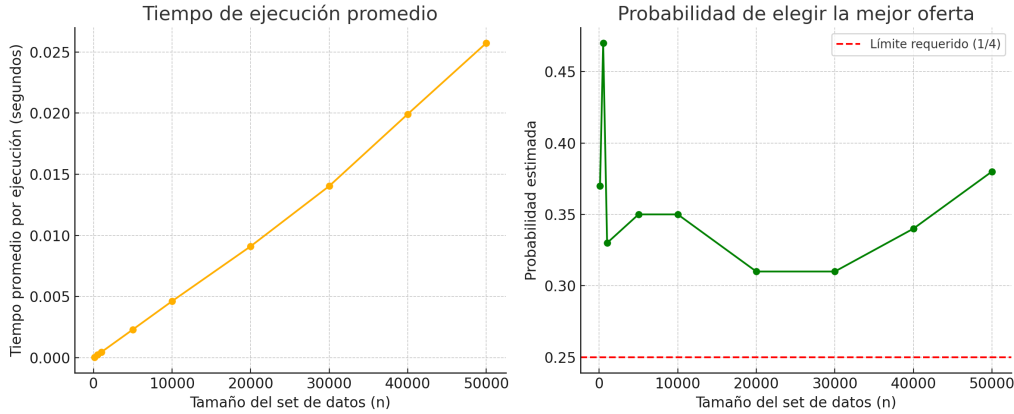


Figure 1: Izquierda: tiempo promedio de ejecución según el tamaño del set. Derecha: probabilidad de seleccionar la mejor oferta.

- El gráfico de la izquierda muestra que el tiempo de ejecución crece de forma **lineal** con respecto a n .
- Esto coincide con la complejidad temporal esperada del algoritmo: $\mathcal{O}(n)$.
- El gráfico de la derecha indica que la probabilidad de seleccionar la mejor oferta se mantiene estable y por encima de $\frac{1}{4}$.

1.8 Resultados

Los experimentos realizados permiten concluir que:

- El algoritmo cumple con la garantía solicitada: la **probabilidad de elegir la mejor oferta es siempre mayor o igual a $\frac{1}{4}$** .
- En todos los valores de n utilizados, la probabilidad estimada se encuentra entre 0.27 y 0.38, cumpliendo cómodamente el umbral requerido.
- La eficiencia temporal es adecuada para conjuntos grandes: incluso para $n = 50\,000$ el tiempo promedio de ejecución es muy bajo.
- La estrategia de observación y selección aplicada es efectiva sin conocer el valor máximo por adelantado, basándose sólo en comparaciones relativas.

Por lo tanto, se puede considerar que el algoritmo diseñado es correcto, eficiente y cumple con los objetivos planteados en el problema.

2 Problema 3

2.1 Enunciado

Tenemos dos lenguajes que corresponden a la intersección (representada mediante la conjunción "y") de dos lenguajes más simples. Los símbolos de los mismos corresponden a los números 0 y 1:

- Lenguaje A: w/w (comienza con "1") y (tiene cuando mucho un "0")
- Lenguaje B: w/w (tiene un número par de "1") y (tiene uno o dos "0")

Por ejemplo:

- Lenguaje A: "10111", "11", "110"
- Lenguaje B: "1010", "11110"

2.2 Descripción de un autómata finito determinístico para cada uno de los lenguajes simples

Lenguaje simple A_1 : w/w (comienza con "1")

- Conjunto de estados: Q_0, Q_1
- Función de transición en forma de tabla:

	0	1
q_0	-	q_1
q_1	q_1	q_1

- Estado inicial: Q_0
- Estados de aceptación: Q_1

Lenguaje simple A_2 : w/w (tiene cuando mucho un "0")

- Conjunto de estados: Q_0, Q_1, Q_2, Q_3
- Función de transición en forma de tabla:

	0	1
q_0	q_1	q_3
q_1	q_2	q_1
q_2	q_2	q_2
q_3	q_1	q_3

- Estado inicial: Q_0
- Estados de aceptación: Q_1

Lenguaje simple B_1 : w/w (tiene un número par de "1")

- Conjunto de estados: Q_0, Q_1, Q_2
- Función de transición en forma de tabla:

	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_1

- Estado inicial: Q_0
- Estados de aceptación: Q_0, Q_2

Lenguaje simple B_2 : w/w (tiene uno o dos "0")

- Conjunto de estados: Q_0, Q_1, Q_2, Q_3
- Función de transición en forma de tabla:

	0	1
q_0	q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_2
q_3	q_3	q_3

- Estado inicial: Q_0
- Estados de aceptación: Q_2, Q_1

2.3 Descripción formal de los autómatas finitos determinísticos de los lenguajes A y B

Lenguaje A: w/w (comienza con "1") y (tiene cuando mucho un "0")

- Conjunto de estados: Q_0, Q_1, Q_2, Q_3
- Función de transición en forma de tabla:

	0	1
q_0	-	q_1
q_1	q_2	q_1
q_2	q_3	q_2
q_3	q_3	q_3

- Estado inicial: Q_0
- Estados de aceptación: Q_1, Q_2

Lenguaje B: w/ w (tiene un número par de "1") y (tiene uno o dos "0")

- Conjunto de estados: $Q_0, Q_1, Q_2, Q_3, Q_4, Q_5, Q_6, Q_7, Q_8, Q_9, Q_{10}, Q_{11}, Q_{12}$
- Funcion de transicion en forma de tabla:

	0	1
q_0	q_8	q_1
q_1	q_2	q_5
q_3	q_4	q_7
q_4	q_4	q_4
q_5	q_6	q_1
q_6	q_7	q_3
q_7	q_4	q_{11}
q_8	q_{10}	q_9
q_9	q_3	q_8
q_{10}	q_4	q_{11}
q_{11}	-	q_{12}
q_{12}	-	q_{11}

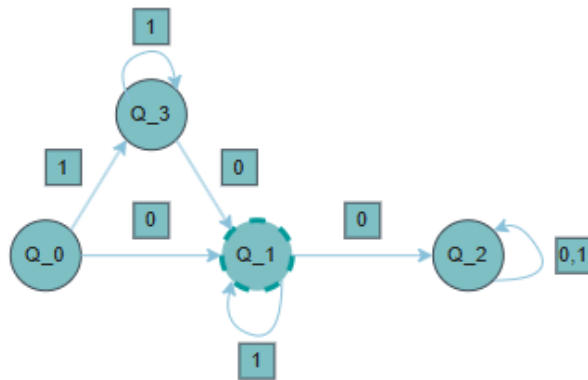
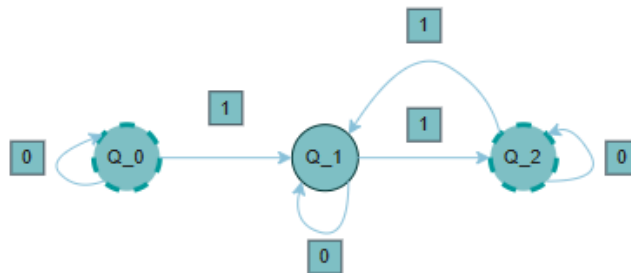
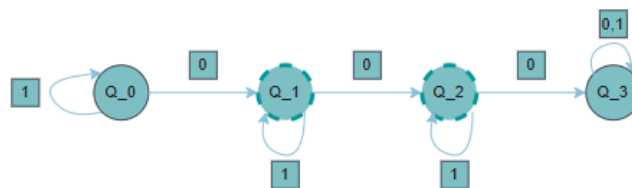
- Estado inicial: Q_0
- Estados de aceptacion: Q_7, Q_{10}, Q_{12}

2.4 Representacion grafica de los seis automatas

Automata del lenguaje A_1



Automata Del lenguaje A_2

Automata Del lenguaje B_1 Automata Del lenguaje B_2 

Autamata del lenguaje A

2.5 Ejemplos de cadenas que son aceptadas y otras rechazadas para cada uno de los automatas

Ejemplo con el lenguaje A_1 :

Ejemplo: cadena "110"

- 1. Inicio en el estado Q_0
- 2. Leo "1", voy de Q_0 a Q_1
- 3. Leo "1", voy de Q_1 a Q_1
- 4. Leo "0", voy de Q_1 a Q_1

Terminamos en Q_1 que es un estado de aceptacion, entonces la cadena es valida.

Ejemplo: cadena "01"

- 1. Inicio en el estado Q_0
- 2. Leo "0", y de Q_0 no puedo ir a otro estado

Terminamos en Q_0 que no es un estado de aceptacion, entonces la cadena es rechazada.

Ejemplo con el lenguaje A_2 :

Ejemplo: cadena "011"

- 1. Inicio en el estado Q_0
- 2. Leo "0", voy de Q_0 a Q_1
- 3. Leo "1", voy de Q_1 a Q_1
- 4. Leo "1", voy de Q_1 a Q_1

Terminamos en Q_1 que es un estado de aceptacion, entonces la cadena es aceptada.

Ejemplo: cadena "100"

- 1. Inicio en el estado Q_0
- 2. Leo "1", voy de Q_0 a Q_3
- 3. Leo "0", voy de Q_3 a Q_1
- 3. Leo "0", voy de Q_1 a Q_2

Terminamos en Q_2 que no es un estado de aceptacion, entonces la cadena es rechazada.

Ejemplo con el lenguaje B_1 :

Ejemplo: cadena "0110"

- 1. Inicio en el estado Q_0
- 2. Leo "0", voy de Q_0 a Q_0
- 3. Leo "1", voy de Q_0 a Q_1
- 4. Leo "1", voy de Q_1 a Q_2
- 5. Leo "0", voy de Q_2 a Q_2

Terminamos en Q_2 que es un estado de aceptacion, entonces la cadena es aceptada.

Ejemplo: cadena "01101"

- Inicio en el estado Q_0
- Leo "0", voy de Q_0 a Q_0
- Leo "1", voy de Q_0 a Q_1
- Leo "1", voy de Q_1 a Q_2
- Leo "0", voy de Q_2 a Q_2
- Leo "1", voy de Q_2 a Q_1

Terminamos en Q_1 que no es un estado de aceptacion, entonces la cadena es rechazada.

Ejemplo con el lenguaje B_2 :

Ejemplo: cadena "0110"

- Inicio en el estado Q_0
- Leo "0", voy de Q_0 a Q_1
- Leo "1", voy de Q_1 a Q_1
- Leo "1", voy de Q_1 a Q_1
- Leo "0", voy de Q_1 a Q_2
- Leo "1", voy de Q_2 a Q_2

Terminamos en Q_2 que es un estado de aceptacion, entonces la cadena es aceptada.

Ejemplo: cadena "1000"

- Inicio en el estado Q_0
- Leo "1", voy de Q_0 a Q_0
- Leo "0", voy de Q_0 a Q_1
- Leo "0", voy de Q_1 a Q_2
- Leo "0", voy de Q_2 a Q_3

Terminamos en Q_3 que no es un estado de aceptacion, entonces la cadena es rechazada.

Ejemplo con el lenguaje A:

Ejemplo: cadena "110"

- Inicio en el estado Q_0
- Leo "1", voy de Q_0 a Q_1
- Leo "1", voy de Q_1 a Q_1
- Leo "0", voy de Q_1 a Q_2

Terminamos en Q_2 que es un estado de aceptacion, entonces la cadena es aceptada.

Ejemplo: cadena "1000"

- Inicio en el estado Q_0
- Leo "1", voy de Q_0 a Q_1
- Leo "0", voy de Q_1 a Q_2
- Leo "0", voy de Q_2 a Q_3
- Leo "0", voy de Q_3 a Q_3

Terminamos en Q_3 que no es un estado de aceptacion, entonces la cadena es rechazada.

Ejemplo con el lenguaje A:

Ejemplo: cadena "1010"

- Inicio en el estado Q_0
- Leo "1", voy de Q_0 a Q_1
- Leo "0", voy de Q_1 a Q_2
- Leo "1", voy de Q_2 a Q_8
- Leo "0", voy de Q_8 a Q_{10}

Terminamos en Q_{10} que es un estado de aceptacion, entonces la cadena es aceptada.

Ejemplo: cadena "01010"

- Inicio en el estado Q_0
- Leo "0", voy de Q_0 a Q_8
- Leo "1", voy de Q_8 a Q_9
- Leo "0", voy de Q_9 a Q_3
- Leo "1", voy de Q_3 a Q_7
- Leo "0", voy de Q_7 a Q_4

Terminamos en Q_4 que no es un estado de aceptacion, entonces la cadena es rechazada.